

Employee Payroll Management System

Quản lý nhân viên & tính lương tháng — Java Console & OOP (kế thừa, đa hình)

Mã đề: J1.L.P0037 • LOC: 450 • Học kỳ SU26

GVHD: Lê Võ Minh Thư

BÀI GIẢNG & TIÊU CHÍ CHẤM

RBL Insight Framework · Computational Thinking · AI Reflection (30%)

Mục lục

1 Khung RBL Insight, Tư duy Tính toán và Định hướng học tập	2
1.1 Vì sao LAB211 không chỉ là "viết cho chạy"	2
1.2 Research-Based Learning (RBL) là gì	2
1.3 Computational Thinking: bốn trụ cột	2
1.4 Cấu trúc báo cáo CT bắt buộc	3
1.5 Chuẩn đầu ra & trực năng lực của tài liệu	3
1.6 Hướng dẫn viết báo cáo & trình bày	4
1.7 Tư duy về độ phức tạp & trường hợp biên	4
2 Phân tích đề bài Employee Payroll	5
2.1 Bối cảnh	5
2.2 Thực thể Employee	5
2.3 Ràng buộc (Validation Rules)	5
2.4 Công thức tính lương (để không cho sẵn — ta định nghĩa hợp lý)	5
2.5 Chức năng	6
3 Áp dụng Computational Thinking	7
3.1 Decomposition — Phân rã	7
3.1.1 Theo dữ liệu	7
3.1.2 Theo chức năng	7
3.1.3 Theo class (OOP View)	7
3.2 Pattern Recognition — Nhận diện mẫu	7
3.3 Abstraction — Trừu tượng hoá	7
3.4 Algorithm Design — Thiết kế thuật toán	7
3.4.1 Workflow: Load có xử lý dòng lỗi (Function 1)	7
3.4.2 Workflow: Tính lương tháng (Function 6)	8
4 Thiết kế hướng đối tượng (OOP)	9
4.1 Use Case & Class Diagram	9
4.2 Inheritance & Polymorphism — trái tim của đề	9
4.2.1 Lớp trừu tượng Employee	9
4.2.2 Bốn lớp con — mỗi vai trò một phụ cấp	10
4.2.3 Factory tạo đối tượng theo role	10
4.2.4 Interface Payable & Encapsulation	11
4.3 Bản đồ bốn nguyên lý OOP → vị trí trong code	11
5 Triển khai chức năng & code mẫu	12
5.1 Menu chính (9 chức năng)	12
5.2 Nạp dữ liệu an toàn (Function 1)	12
5.3 Thêm & cập nhật nhân viên (Functions 2, 3)	13
5.4 Tìm theo thuộc tính & tính lương (Functions 5, 6)	13
5.5 Hiển thị, xoá, lưu, thoát (Functions 7, 4, 8, 9)	14
5.6 Mẫu kết quả (Sample Output)	14
5.7 Từ điển dữ liệu & Flowchart Add	15
5.8 Phiên chạy mẫu đầy đủ	15

6 Tiêu chí chấm (Rubric)	17
6.1 Cấu trúc tổng thể (thang 100, quy đổi 450 LOC)	17
6.2 Chi tiết chức năng (15đ)	17
6.3 Chi tiết CT (20đ) & OOP (25đ)	17
6.4 Worked Example & lỗi mất điểm	18
6.5 AI Reflection (30đ)	18
7 AI Audit Log & AI Reflection — Cách làm và cách chấm	19
7.1 AI Audit Log là gì và <i>không</i> là gì	19
7.2 Core Prompt vs. Auxiliary Prompt	19
7.3 Yêu cầu số lượng (bài Assignment LAB211)	19
7.4 Cấu trúc một entry: 7 phần	19
7.5 Phát hiện Hallucination (bắt buộc)	20
7.6 Tiêu chí chấm AI Reflection (30%)	20
7.7 Dấu hiệu nhận biết khi chấm	21
7.8 Quy định cho sinh viên	21
7.9 AI Literacy — viết prompt hiệu quả	21
7.10 Phương pháp Oral Vivas (vấn đáp)	22
7.11 Bảng phân bố entries theo thành phần CT	22
8 Ví dụ AI Audit Log cho Payroll	23
8.1 Entry ĐẠT — Decision (thiết kế kế thừa)	23
8.2 Entry phát hiện HALLUCINATION (công thức lương)	23
8.3 Entry ĐẠT — Problem-Solving (dòng dữ liệu lỗi)	23
8.4 Entry KÉM (để đối chiếu)	23
8.5 Ngân hàng câu hỏi vấn đáp (Payroll)	24
8.6 Chỉ mục Audit Log mẫu (21 core prompts)	24
Phụ lục — Checklist tự chấm	24

1

Khung RBL Insight, Tư duy Tính toán và Định hướng học tập

1.1 Vì sao LAB211 không chỉ là "viết cho chạy"

Trong môn **LAB211**, một chương trình chạy đúng yêu cầu chỉ là điều kiện *cần*. Điều kiện *đủ* — và là phần tạo ra giá trị học tập thực sự — nằm ở chỗ sinh viên **hiểu vì sao** mình thiết kế như vậy, **chứng minh được** quá trình tư duy, và **kiểm soát được** mọi dòng code (kể cả code do AI gợi ý). Đây chính là tinh thần của **RBL Insight Framework**.

Tuyên ngôn cốt lõi

AI không thay thế sinh viên. AI là trợ lý hỗ trợ tư duy, phân tích, gỡ lỗi và thiết kế để quá trình học tập của sinh viên được thuận lợi. Giá trị được chấm điểm là phần **con người thêm vào** mà AI không tự làm được — gọi là *Human Delta*.

Ba năng lực mà tài liệu này rèn cho người học, đặt cạnh nhau và bổ trợ lẫn nhau:



Từ phân tích vấn đề → thiết kế hướng đối tượng
→ minh bạch và phản biện quá trình dùng AI.

Hình 1: Ba năng lực cốt lõi: Tư duy tính toán, OOP và AI Reflection

1.2 Research-Based Learning (RBL) là gì

RBL — Research-Based Learning là cách học lấy *quá trình tìm tòi, đặt câu hỏi, thử nghiệm và phản biện* làm trung tâm, thay vì chỉ tiếp nhận lời giải có sẵn. Khi kết hợp với **PBL (Project-Based Learning)**, sinh viên học thông qua một dự án thực tế và phải tự trả lời các câu hỏi nghiên cứu nhỏ:

- **Đặt vấn đề:** Yêu cầu thực sự là gì? Ràng buộc nào là cốt lõi?
- **Giả thuyết & phương án:** Có những cách tiếp cận nào? Ưu/nhược điểm?
- **Thử nghiệm & đo lường:** Phương án nào tốt hơn, dựa trên bằng chứng gì?
- **Kết luận & sở hữu quyết định:** Tôi chọn gì, vì sao, và tôi chịu trách nhiệm về lựa chọn đó.

RBL Insight gắn với AI thế nào?

"Insight" ở đây là *bằng chứng tư duy* mà sinh viên để lại trong suốt quá trình làm dự án. Khi sinh viên dùng AI, bằng chứng đó được ghi lại có chọn lọc trong **AI Audit Log** — không phải để "khoe đã hỏi gì" mà để chứng minh *đã suy nghĩ, đã phản biện, đã quyết định*.

1.3 Computational Thinking: bốn trụ cột

Computational Thinking (CT) là phương pháp sư phạm rèn tư duy giải quyết vấn đề theo cách tiếp cận của khoa học máy tính. CT gồm bốn thành phần, và toàn bộ phần "Bài giảng"

của tài liệu này được tổ chức quanh bốn trụ cột đó.



Hình 2: Computational Thinking: bốn trụ cột

- **Decomposition — Phân rã.** Chia bài toán theo *dữ liệu*, theo *chức năng* và theo *lớp (OOP view)*. Mỗi phần nhỏ có thể giải độc lập rồi ghép lại.
- **Pattern Recognition — Nhận diện mẫu.** Tìm điểm chung: các quy luật *validate* (dùng Regular Expression), các thao tác *CRUD* lặp lại giữa các thực thể → tạo template/hàm dùng chung.
- **Abstraction — Trừu tượng hoá.** Với mỗi đối tượng, xác định thuộc tính và hành vi cốt lõi; với mỗi chức năng, chỉ quan tâm *input* → *xử lý* → *output*, bỏ qua chi tiết không cần thiết.
- **Algorithm Design — Thiết kế thuật toán.** Mô tả *Operation Workflow* bằng pseudocode/flowchart *trước khi viết code Java*.

Quy tắc vàng khi áp dụng CT

Luôn đi theo trình tự: **hiểu yêu cầu** → **phân rã** → **nhận diện mẫu** → **trừu tượng hoá thành lớp** → **viết pseudocode** → **rồi mới code**. "Code trước, nghĩ sau" là nguyên nhân số một của lỗi logic và của những bài làm không giải thích được.

1.4 Cấu trúc báo cáo CT bắt buộc

Phần trình bày/báo cáo của sinh viên cần bám theo bộ khung sau (giảng viên dùng chính khung này để đối chiếu khi chấm CT):

1. Introduction — giới thiệu bài toán và mục tiêu.
2. Problem Analysis — phân tích yêu cầu, dữ liệu, ràng buộc.
3. Computational Thinking: 3.1 Decomposition, 3.2 Pattern Recognition, 3.3 Abstraction, 3.4 Algorithm Design.
4. OOP Design (UML) — use case, class diagram (và sequence nếu có).
5. Flowchart — sơ đồ luồng cho các chức năng chính.
6. Conclusion — tổng kết, hạn chế, hướng mở rộng.

1.5 Chuẩn đầu ra & trực năng lực của tài liệu

Tài liệu được thiết kế bám sát các chuẩn đầu ra của học phần LAB211: vận dụng tư duy tính toán để phân tích bài toán, áp dụng lập trình hướng đối tượng để thiết kế giải pháp, và sử dụng công cụ AI một cách có kiểm chứng. Bốn trực năng lực được rèn luyện và đánh giá xuyên suốt như sau:

Trực năng lực → phần tài liệu tương ứng	
Tư duy CT	Hiểu & áp dụng 4 thành phần vào dự án cụ thể (Chương CT).
OOP vững	Xác định class, đóng gói, kế thừa, đa hình (Chương OOP).
Chất lượng dùng AI	Kiểm chứng, hiểu trước khi dùng, chỉnh sửa output (AI Audit Log).
Phản biện (AI Reflection)	Human Delta, phát hiện hallucination, tự đánh giá lại.
AI Literacy	Nguyên tắc dùng GenAI, viết prompt hiệu quả, hiểu rủi ro & đạo đức.

20đ

Computational Thinking

25đ

OOP Design & Quality

30đ

AI Reflection

25đ

Chức năng + Code style

(Thang quy đổi 100 điểm — chi tiết ở chương Tiêu chí chấm.)

1.6 Hướng dẫn viết báo cáo & trình bày

Báo cáo tốt là báo cáo *chứng minh tư duy*, không phải liệt kê code. Bộ khung mẫu để sinh viên triển khai sáu mục đã nêu:

Mục	Nội dung cần có (gợi ý 0.5-1 trang/mục)
1. Introduction	Bài toán là gì, ai dùng, mục tiêu, phạm vi.
2. Problem Analysis	Dữ liệu vào/ra, ràng buộc, các trường hợp biên (edge cases).
3. Computational Thinking	4 tiểu mục: sơ đồ phân rã, bảng mẫu lặp, bảng thuộc tính trừu tượng, pseudo-code/flowchart.
4. OOP Design (UML)	Use case + class diagram + (tuỳ chọn) sequence; giải thích quan hệ.
5. Flowchart	Sơ đồ luồng cho 2-3 chức năng phức tạp nhất.
6. Conclusion	Đã đạt gì, hạn chế, hướng mở rộng, bài học rút ra.

Chuẩn bị trình bày (Oral / Demo)

Chuẩn bị một *kịch bản demo 5 phút*: (1) chạy menu, (2) thêm một bản ghi hợp lệ và một bản ghi bị chặn (để chứng minh validate), (3) một chức năng truy vấn, (4) thống kê, (5) lưu & thoát có nhắc lưu. Vừa thao tác vừa giải thích vì *sao* code xử lý như vậy.

1.7 Tư duy về độ phức tạp & trường hợp biên

Một phần quan trọng của CT là *lường trước* cái gì có thể sai. Trước khi code mỗi chức năng, hãy liệt kê edge cases:

Nhóm	Trường hợp biên cần xử lý
Danh sách rỗng	Hiển thị/tìm/thống kê khi chưa có dữ liệu → thông báo phù hợp, không crash.
Trùng khoá	Thêm bản ghi có ID đã tồn tại → chặn.
Khoá ngoại sai	Tham chiếu tới mã không tồn tại → báo lỗi, yêu cầu nhập lại.
Nhập sai kiểu	Nhập chữ khi cần số, vượt khoảng cho phép → vòng lặp nhập lại.
Hủy thao tác	Người dùng chọn "N" khi xác nhận xoá → giữ nguyên dữ liệu.
Thoát chưa lưu	Có thay đổi mà chưa lưu → nhắc lưu (cờ dirty).

Liên hệ độ phức tạp

Khi danh sách lớn, lựa chọn cấu trúc dữ liệu ảnh hưởng hiệu năng: tra cứu theo khoá nên dùng `HashMap` ($O(1)$) thay vì duyệt `List` ($O(n)$); sắp xếp đa tiêu chí nên dùng `Comparator chaining`. Những lựa chọn này chính là *core prompts* đáng ghi vào Audit Log.

2

Phân tích đề bài Employee Payroll

2.1 Bối cảnh

Một công ty quy mô vừa cần ứng dụng Java console để quản lý thông tin nhân viên và *tính lương tháng*. Mỗi nhân viên có thông tin cá nhân, thông tin công việc và dữ liệu lương. Hệ thống lưu hồ sơ ở tệp văn bản, hỗ trợ tính lương, tìm kiếm, cập nhật và báo cáo.

Điểm khác biệt quan trọng

Đề này có *cơ hội kế thừa & đa hình rõ rệt*: nhiều **vai trò** (Developer, Tester, Manager, HR) có cách cộng phụ cấp khác nhau → thiết kế `Employee` trừu tượng + lớp con với `calculateSalary()` đa hình. Ngoài ra, dữ liệu nạp lên *có thể chứa dòng lỗi* cần xử lý an toàn.

2.2 Thực thể Employee

Listing 2.1. employees.txt: <id, name, role, baseSalary, workingDays, bonus, status>

```
E004, David Miller, Developer, 1300, 21, 250, active
E005, Emma Wilson, Tester, 950, 20, 100, active
E008, Henry Anderson, Manager, 2200, 19, 600, active
E009, Ivy Thomas, Tester, 980, 18, 120, inactive
```

2.3 Ràng buộc (Validation Rules)

Trường	Quy tắc	Mẫu hợp lệ
Employee ID	Không rỗng; duy nhất; định dạng E+3 chữ số.	E001
Name	Không rỗng.	David Miller
Role	∈ {Developer, Tester, Manager, HR}.	Developer
Base Salary	Số dương.	1300
Working Days	Số nguyên 0–26.	21
Bonus	≥ 0.	250
Status	active hoặc inactive.	active

2.4 Công thức tính lương (để không cho sẵn — ta định nghĩa hợp lý)

Định nghĩa lương tháng dùng trong tài liệu này

$$\text{salary} = \underbrace{\frac{\text{baseSalary}}{26} \times \text{workingDays}}_{\text{lương theo ngày công}} + \text{bonus} + \underbrace{\text{roleAllowance()}}_{\text{phụ cấp theo vai trò (đa hình)}}$$

Trong đó `roleAllowance()` khác nhau theo vai trò (xem Chương OOP). **Bảng lương tháng (Function 6)** chỉ cộng lương của nhân viên `active`.

Đây là một quyết định thiết kế — phải ghi Audit Log

Vì để *không* quy định công thức, sinh viên phải tự định nghĩa và *giải thích* (một core prompt Decision). Không được để AI “bịa” công thức rồi dùng mà không kiểm chứng (xem hallucination ở §8).

2.5 Chức năng

#	Chức năng	Mô tả
1	Load from file	Đọc <code>employees.txt</code> ; bỏ qua/báo dòng lỗi an toàn.
2	Add a new employee	Nhập, validate, tạo đúng lớp con theo role, lưu.
3	Update employee	Cập nhật role, salary, bonus, status (Enter giữ cũ).
4	Remove employee by ID	Xoá theo ID; báo nếu không tồn tại.
5	Search by attribute	Tìm theo ID / name / role / status.
6	Calculate monthly payroll	Tổng lương cho các nhân viên <code>active</code> .
7	Display employee list	Hiển thị toàn bộ nhân viên (bảng).
8	Save data to file	Ghi dữ liệu trở lại tệp.
9	Quit program	Xác nhận lưu trước khi thoát.

3

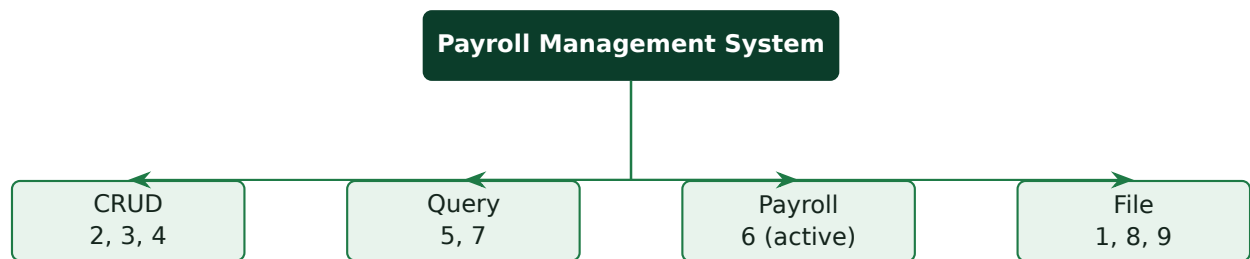
Áp dụng Computational Thinking

3.1 Decomposition — Phân rã

3.1.1 Theo dữ liệu

Một thực thể **Employee** với 7 trường; phân hoá theo *vai trò* (4 loại) → gợi ý phân cấp lớp. Đọc/ghi `employees.txt`.

3.1.2 Theo chức năng



Hình 3: Theo chức năng

3.1.3 Theo class (OOP View)

Employee (abstract) + Developer, Tester, Manager, HR (model); EmployeeFactory (tạo theo role); Employees (business: CRUD, search, payroll, I/O); Inputter, Validator (tools); Main (dispatcher).

3.2 Pattern Recognition — Nhận diện mẫu

- **Validate bằng Regex:** `\d{3}` cho ID; role/status dùng tập giá trị hợp lệ.
- **CRUD template:** Add/Update/Remove/Search lặp trên `List<Employee>`.
- **Đa hình thay cho if-else theo role:** thay vì `if(role=="Manager")...` rườm rà, đưa hành vi khác nhau vào lớp con (`roleAllowance()`, `getRole()`).

3.3 Abstraction — Trừu tượng hoá

Thuộc tính cốt lõi chung: `id`, `name`, `baseSalary`, `workingDays`, `bonus`, `status`. Hành vi cốt lõi chung: `calculateSalary()` (khung), `isActive()`, `toString()`. Phần *khác biệt* theo vai trò được trừu tượng hoá thành `roleAllowance()` (abstract).

3.4 Algorithm Design — Thiết kế thuật toán

3.4.1 Workflow: Load có xử lý dòng lỗi (Function 1)

Listing 3.1. Pseudocode — nạp an toàn

```

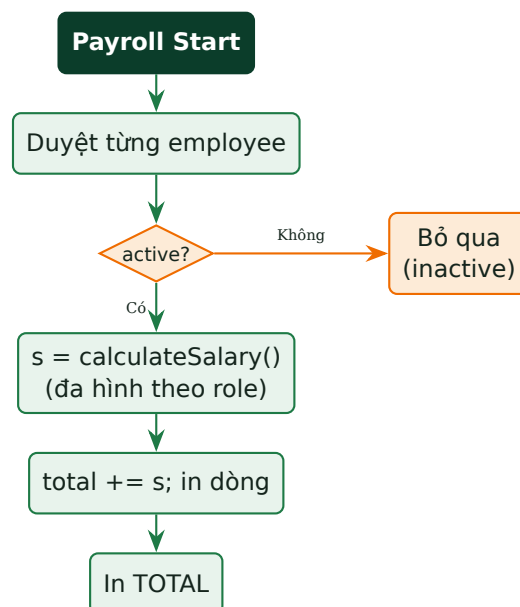
FUNCTION load(path):
  ok = 0; bad = 0
  FOREACH line IN file:
    parts = split(line, ",")
    IF parts.size != 7 THEN bad++; CONTINUE
    IF NOT validateAll(parts) THEN bad++; CONTINUE // sai format/role/range
    emp = EmployeeFactory.create(parts) // dùng lớp con theo role
    IF employees.exists(emp.id) THEN bad++; CONTINUE // trùng id
    employees.add(emp); ok++
  
```

```
PRINT "Loaded " + ok + " records, skipped " + bad + " invalid lines."
END
```

3.4.2 Workflow: Tính lương tháng (Function 6)

Listing 3.2. Pseudocode — tổng lương active

```
FUNCTION monthlyPayroll():
    total = 0
    FOREACH e IN employees:
        IF e.isActive() THEN
            s = e.calculateSalary() // đa hình: mọi role tính khác nhau
            PRINT e.id, e.getRole(), format(s)
            total += s
    PRINT "TOTAL (active only): " + format(total)
END
```

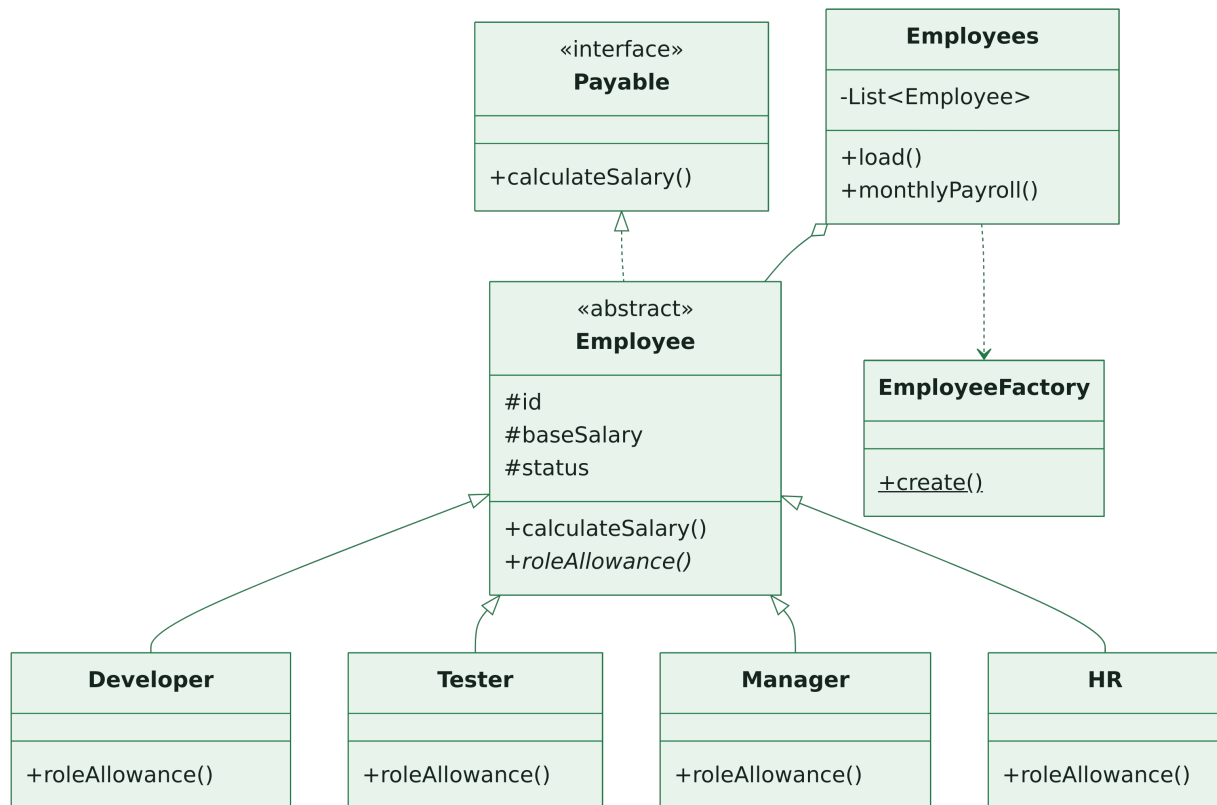


Hình 4: Workflow: Tính lương tháng (Function 6)

4

Thiết kế hướng đối tượng (OOP)

4.1 Use Case & Class Diagram



Hình 5: Use Case & Class Diagram

- Employee (abstract) là gốc; Developer/Tester/Manager/HR kế thừa và override roleAllowance().
- Employees *has-a* List<Employee>; EmployeeFactory tạo đúng lớp con theo role.
- Payable (interface) khai báo calculateSalary(); minh hoạ đa hình qua List<Employee>.

4.2 Inheritance & Polymorphism — trái tim của đề

4.2.1 Lớp trừu tượng Employee

Listing 4.1. Employee: khung tính lương + roleAllowance() trừu tượng

```

1 public abstract class Employee implements Serializable, Payable {
2     protected String id, name, status; // status: active / inactive
3     protected double baseSalary, bonus;
4     protected int workingDays;
5
6     protected Employee(String id, String name, double baseSalary,
7                         int workingDays, double bonus, String status) {
8         this.id = id; this.name = name; this.baseSalary = baseSalary;
9         this.workingDays = workingDays; this.bonus = bonus; this.status = status;
10    }
11    public abstract String getRole(); // moi lop con tra ve vai tro
12    public abstract double roleAllowance(); // phu cap khac nhau theo vai tro
  
```

```

13
14     @Override public double calculateSalary() { // KHUNG chung, dung roleAllowance() da hinh
15         double byDays = baseSalary / 26.0 * workingDays;
16         return byDays + bonus + roleAllowance();
17     }
18     public boolean isActive() { return "active".equalsIgnoreCase(status); }
19     public String getId() { return id; }
20     @Override public String toString() {
21         return String.format("%-5s| %-15s| %-10s| %8.0f| %3d| %6.0f| %-8s| %,1f",
22             id, name, getRole(), baseSalary, workingDays, bonus, status, calculateSalary());
23     }
24 }

```

4.2.2 Bốn lớp con — mỗi vai trò một phụ cấp

Listing 4.2. Developer / Tester / Manager / HR override roleAllowance()

```

1 public class Developer extends Employee {
2     public Developer(String id,String n,double b,int d,double bo,String
3         s){super(id,n,b,d,bo,s);}
4     public String getRole(){ return "Developer"; }
5     public double roleAllowance(){ return 0.10 * baseSalary; } // tech allowance 10%
6 }
7 public class Tester extends Employee {
8     public Tester(String id,String n,double b,int d,double bo,String
9         s){super(id,n,b,d,bo,s);}
10    public String getRole(){ return "Tester"; }
11    public double roleAllowance(){ return 0.05 * baseSalary; } // 5%
12 }
13 public class Manager extends Employee {
14     public Manager(String id,String n,double b,int d,double bo,String
15         s){super(id,n,b,d,bo,s);}
16    public String getRole(){ return "Manager"; }
17    public double roleAllowance(){ return 0.20 * baseSalary + 200; } // management allowance
18 }
19 public class HR extends Employee {
20     public HR(String id,String n,double b,int d,double bo,String s){super(id,n,b,d,bo,s);}
21     public String getRole(){ return "HR"; }
22     public double roleAllowance(){ return 0.08 * baseSalary; } // 8%
23 }

```

Vì sao đây là "áp dụng nâng cao" (đạt 10đ Inheritance)?

Khi thêm vai trò mới (vd Designer), chỉ cần *thêm một lớp con*, không sửa vòng tính lương. calculateSalary() gọi roleAllowance() đã định — không có chuỗi if-else theo role. Đây là minh họa kinh điển của *Open/Closed Principle*.

4.2.3 Factory tạo đối tượng theo role

Listing 4.3. EmployeeFactory: ánh xạ role → lớp con

```

1 public class EmployeeFactory {
2     public static Employee create(String id, String name, String role,
3         double base, int days, double bonus, String status) {
4         switch (role.toLowerCase()) {
5             case "developer": return new Developer(id, name, base, days, bonus, status);
6             case "tester": return new Tester(id, name, base, days, bonus, status);
7             case "manager": return new Manager(id, name, base, days, bonus, status);

```

```

8         case "hr": return new HR(id, name, base, days, bonus, status);
9         default: throw new IllegalArgumentException("Unknown role: " + role);
10    }
11 }
12 }

```

4.2.4 Interface Payable & Encapsulation

Listing 4.4. Payable + truy cập có kiểm soát

```

1 public interface Payable { double calculateSalary(); }
2 // Encapsulation: field protected/private; thay doi qua setter co kiểm tra
3 public void setStatus(String s) {
4     if (s.equalsIgnoreCase("active") || s.equalsIgnoreCase("inactive")) this.status = s;
5 }
6 public void setBonus(double b) { if (b >= 0) this.bonus = b; }
7 public void setWorkingDays(int d) { if (d >= 0 && d <= 26) this.workingDays = d; }

```

4.3 Bản đồ bốn nguyên lý OOP → vị trí trong code

Nguyên lý	Thể hiện
Abstraction	Employee abstract + roleAllowance()/getRole() abstract; interface Payable.
Encapsulation	field protected/private + setter kiểm tra (status/bonus/workingDays).
Inheritance	Developer/Tester/Manager/HR extends Employee.
Polymorphism	calculateSalary() dùng roleAllowance() đa hình; duyệt List<Employee>.

5

Triển khai chức năng & code mẫu

5.1 Menu chính (9 chức năng)

Listing 5.1. Vòng lặp menu

```
1 do {
2     showMenu();
3     int c = Inputter.inputInt("Choose 1-9: ", 1, 9);
4     switch (c) {
5         case 1 -> employees.load("employees.txt");
6         case 2 -> employees.add();
7         case 3 -> employees.update();
8         case 4 -> employees.remove();
9         case 5 -> employees.searchByAttribute();
10        case 6 -> employees.monthlyPayroll();
11        case 7 -> employees.display();
12        case 8 -> employees.save("employees.txt");
13        case 9 -> employees.quit();
14    }
15 } while (choice != 9);
```

5.2 Nạp dữ liệu an toàn (Function 1)

Listing 5.2. load(): bỏ qua dòng lỗi, đếm & báo

```
1 public void load(String path) {
2     int ok = 0, bad = 0;
3     try (BufferedReader br = new BufferedReader(new FileReader(path))) {
4         String line;
5         while ((line = br.readLine()) != null) {
6             if (line.isBlank()) continue;
7             String[] p = line.split("\\s*,\\s*");
8             if (p.length != 7) { bad++; continue; }
9             try {
10                 String id = p[0], name = p[1], role = p[2], status = p[6];
11                 double base = Double.parseDouble(p[3]);
12                 int days = Integer.parseInt(p[4]);
13                 double bonus = Double.parseDouble(p[5]);
14                 if (!Validator.isEmployeeId(id) || name.isEmpty() || !Validator.isRole(role)
15                     || base <= 0 || days < 0 || days > 26 || bonus < 0
16                     || !Validator.isStatus(status) || exists(id)) { bad++; continue; }
17                 list.add(EmployeeFactory.create(id, name, role, base, days, bonus, status));
18                 ok++;
19             } catch (NumberFormatException ex) { bad++; } // erroneous numeric data
20         }
21     } catch (IOException e) { System.out.println("Cannot read file: " + e.getMessage());
22         return; }
23     System.out.println("Loaded " + ok + " records, skipped " + bad + " invalid line(s).");
24 }
```

Xử lý "dòng dữ liệu lỗi" — yêu cầu tường minh của đề

Đề ghi rõ "data reading process may encounter lines containing erroneous data". Vì vậy load phải *không crash*: bọc try-catch cho parse số, kiểm tra đủ 7 trường, validate từng trường, và *báo cáo* số dòng bị bỏ. Đây là một edge case kinh điển hay bị AI bỏ sót.

5.3 Thêm & cập nhật nhân viên (Functions 2, 3)

Listing 5.3. add() and update() — tạo lớp con theo role; Enter giữ cũ

```

1 public void add() {
2     String id = Inputter.inputLoop("ID (Exxx): ", Validator.ID_REGEX, "Format must be E + 3
        digits");
3     if (exists(id)) { System.out.println("This employee ID already exists!"); return; }
4     String name = Inputter.inputNonEmpty("Name: ");
5     String role = Inputter.inputInSet("Role (Developer/Tester/Manager/HR): ",
        Validator.ROLES);
6     double base = Inputter.inputPositiveDouble("Base salary: ");
7     int days = Inputter.inputInt("Working days (0-26): ", 0, 26);
8     double bonus = Inputter.inputNonNegativeDouble("Bonus: ");
9     String status = Inputter.inputInSet("Status (active/inactive): ", Validator.STATUSSES);
10    list.add(EmployeeFactory.create(id, name, role, base, days, bonus, status));
11    dirty = true; System.out.println("Employee added.");
12 }
13 public void update() {
14     String id = Inputter.inputLoop("ID: ", Validator.ID_REGEX, "Format must be E + 3
        digits");
15     Employee e = findById(id);
16     if (e == null) { System.out.println("This employee does not exist!"); return; }
17     String bonusStr = Inputter.inputOptional("Bonus [" + e.getBonus() + "]: ");
18     if (!bonusStr.isEmpty()) e.setBonus(Double.parseDouble(bonusStr));
19     String statusStr = Inputter.inputOptional("Status [" + e.getStatus() + "]: ");
20     if (!statusStr.isEmpty()) e.setStatus(statusStr);
21     // role/salary tương tự; doi role -> tạo lại dung lớp con (xem ghi chú)
22     dirty = true; System.out.println("Updated successfully.");
23 }

```

Đổi role khi Update

Vì mỗi role là một *lớp con* khác nhau, đổi role nên *tạo lại* đối tượng bằng EmployeeFactory (giữ nguyên các trường khác) thay vì gán chuỗi role — một điểm tinh tế đáng đưa vào Audit Log (Decision).

5.4 Tìm theo thuộc tính & tính lương (Functions 5, 6)

Listing 5.4. searchByAttribute() and monthlyPayroll()

```

1 public void searchByAttribute() {
2     System.out.println("Search by: 1.ID 2.Name 3.Role 4.Status");
3     int k = Inputter.inputInt("Choose: ", 1, 4);
4     String kw = Inputter.inputNonEmpty("Keyword: ").toLowerCase();
5     List<Employee> found = new ArrayList<>();
6     for (Employee e : list) {
7         String field = switch (k) {
8             case 1 -> e.getId(); case 2 -> e.getName();
9             case 3 -> e.getRole(); default -> e.getStatus();
10        };

```

```

11     if (field.toLowerCase().contains(kw)) found.add(e);
12 }
13 if (found.isEmpty()) System.out.println("No employee matches.");
14 else found.forEach(System.out::println);
15 }
16 public void monthlyPayroll() {
17     double total = 0; boolean any = false;
18     System.out.printf("%-5s| %-10s| %s\n", "ID", "Role", "Salary");
19     for (Employee e : list) if (e.isActive()) {
20         double s = e.calculateSalary(); // polymorphic
21         System.out.printf("%-5s| %-10s| %,1f\n", e.getId(), e.getRole(), s);
22         total += s; any = true;
23     }
24     if (!any) System.out.println("No active employee.");
25     else System.out.printf("TOTAL (active only): %,1f\n", total);
26 }

```

5.5 Hiển thị, xóa, lưu, thoát (Functions 7, 4, 8, 9)

Listing 5.5. display(), remove(), save(), quit()

```

1 public void display() {
2     if (list.isEmpty()) { System.out.println("The employee list is empty."); return; }
3     System.out.printf("%-5s| %-15s| %-10s| %8s| %3s| %6s| %-8s| %s\n",
4         "ID", "Name", "Role", "Base", "Day", "Bonus", "Status", "Salary");
5     list.forEach(System.out::println);
6 }
7 public void remove() {
8     String id = Inputter.inputLoop("ID: ", Validator.ID_REGEX, "Format must be E + 3
9         digits");
10    Employee e = findById(id);
11    if (e == null) { System.out.println("This employee does not exist!"); return; }
12    list.remove(e); dirty = true; System.out.println("Removed.");
13 }
14 public void save(String path) {
15     try (PrintWriter pw = new PrintWriter(new FileWriter(path))) {
16         for (Employee e : list)
17             pw.printf("%s, %s, %s, %.0f, %d, %.0f, %s\n", e.getId(), e.getName(), e.getRole(),
18                 e.getBaseSalary(), e.getWorkingDays(), e.getBonus(), e.getStatus());
19         dirty = false; System.out.println("Data saved to " + path + ".");
20     } catch (IOException ex) { System.out.println("Save failed: " + ex.getMessage()); }
21 }
22 public void quit() {
23     if (dirty) {
24         String c = Inputter.inputLoop("Save before exit? (Y/N): ", "^[YyNn]$", "Y or N");
25         if (c.equalsIgnoreCase("Y")) save("employees.txt");
26     }
27 }

```

5.6 Mẫu kết quả (Sample Output)

Listing 5.6. Load + payroll

```

Choose 1-9: 1
Loaded 9 records, skipped 1 invalid line(s).

Choose 1-9: 6

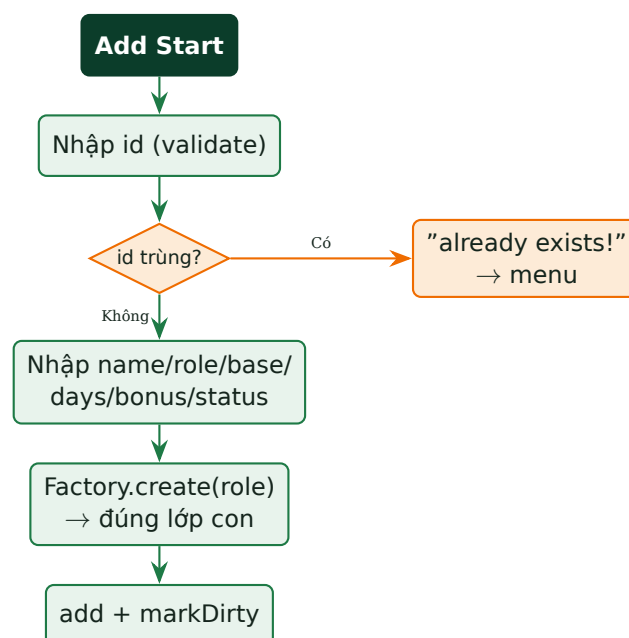
```



```
ID | Role | Salary
E004 | Developer | 1,580.0
E008 | Manager | 3,505.4
(E009 inactive -> skipped)
TOTAL (active only): 12,7xx.x
```

5.7 Từ điển dữ liệu & Flowchart Add

Thuộc tính	Kiểu	Ý nghĩa & ràng buộc
id	String	Khoá chính E\d{3}, duy nhất.
name	String	Không rỗng.
role	(lớp con)	Developer/Tester/Manager/HR.
baseSalary	double	> 0.
workingDays	int	0-26.
bonus	double	≥ 0.
status	String	active / inactive.



Hình 6: Từ điển dữ liệu & Flowchart Add

5.8 Phiên chạy mẫu đầy đủ

Listing 5.7. Một phiên thao tác minh họa

```
Choose 1-9: 1
Loaded 9 records, skipped 1 invalid line(s).

Choose 1-9: 2
ID (Exxx): E01 -> Format must be E + 3 digits
ID (Exxx): E014
Name: Olivia Reed
Role (Developer/Tester/Manager/HR): Developer
Base salary: 1200
Working days (0-26): 30 -> Please enter a number in [0,26]
Working days (0-26): 22
Bonus: 200
Status (active/inactive): active
```

Employee added.

Choose 1-9: 6

ID | Role | Salary

E004 | Developer | 1,580.0

E014 | Developer | 1,535.4

E008 | Manager | 3,505.4

TOTAL (active only): 14,9xx.x

Choose 1-9: 9

Save before exit? (Y/N): Y

Data saved to employees.txt.

6

Tiêu chí chấm (Rubric)

6.1 Cấu trúc tổng thể (thang 100, quy đổi 450 LOC)

Khối	Điểm	Ghi chú
Computational Thinking (CT)	20	Decomp 5 + Pattern 3 + Abstraction 6 + Algorithm 6
AI Reflection — CT	10	thuộc 30%
Áp dụng OOP	25	Class 6 + Encapsulation 4 + Ctor/toString/equals 5 + Inheritance 10
AI Reflection — OOP	10	thuộc 30%
Chức năng (CRUD + File I/O + Payroll)	15	chi tiết bên dưới
AI Reflection — Functional	10	thuộc 30%
Code style & báo cáo	10	Naming 4 + Clean 4 + Report 2
Tổng	100	AI Reflection cộng dồn = 30%

Đề này "ăn điểm" ở Inheritance

Khối Inheritance/polymorphism (10đ) là nơi đề Payroll dễ ghi điểm nhất nhờ thiết kế Employee abstract + 4 lớp con. Ngược lại, nếu sinh viên dùng một lớp Employee duy nhất với if-else theo role, khối này bị trừ nặng.

6.2 Chi tiết chức năng (15đ)

Chức năng	Điểm	Chức năng	Điểm
Display Employee (table)	2	Tính lương theo Payroll	2
Thêm Employee (validation)	2.5	Xoá Employee	2
Tìm kiếm & cập nhật Employee	2.5	Lưu dữ liệu vào file	2
Lọc Employee theo Attribute	2		

(Tổng 15đ; thang 100, quy đổi theo 450 LOC.)

6.3 Chi tiết CT (20đ) & OOP (25đ)

CT	Điểm	OOP	Điểm
Decomposition	5	Định nghĩa class (Employee, lớp con...)	6
Pattern Recognition	3	Encapsulation	4
Abstraction	6	Constructor/toString/equals	5
Algorithm Design	6	Kế thừa/interface/polymorphism	10

6.4 Worked Example & lỗi mất điểm

Khối	Điểm	Nhận xét
CT (20)	17	Phân rã & abstraction tốt; flowchart payroll rõ; thiếu pseudocode Update.
OOP (25)	24	4 lớp con + đa hình <code>calculateSalary()</code> ; encapsulation tốt.
Functional (15)	13	9 chức năng chạy; payroll chỉ tính active đúng; load báo dòng lỗi.
Code style (10)	8	Tên rõ; load hơi dài nên tách <code>validateLine()</code> .
AI Reflection (30)	26	21 core prompts; 2 hallucination (gồm công thức lương); evidence đầy đủ.
Tổng thô	88/100	Tốt.

Lỗi thường gặp	Khối bị trừ
Một lớp Employee + if-else theo role (không kế thừa).	OOP (Inheritance 10đ)
Payroll cộng cả nhân viên inactive.	Functional (Payroll)
load crash khi gặp dòng lỗi (không try-catch).	Functional (Load) / Code style
Công thức lương lấy từ AI mà không kiểm chứng/giải thích.	AI Reflection (Critical Thinking)
Không xử lý <code>workingDays</code> ngoài 0-26.	Functional / Validation

6.5 AI Reflection (30đ)

Áp dụng bộ tiêu chí ở §7.6. Lưu ý đặc thù: đề *không* cho công thức lương nên *ít nhất một* core prompt phải là quyết định/kiểm chứng công thức (kèm phát hiện nếu AI bịa quy tắc).

AI Audit Log & AI Reflection — Cách làm và cách chấm

7.1 AI Audit Log là gì và *không* là gì

KHÔNG phải là

- Nơi copy-paste *toàn bộ* lịch sử chat với ChatGPT/Gemini/Claude.
- Bộ sưu tập các prompt vặt ("khai báo mắng thế nào?").
- Bảng chứng "đã dùng AI" một cách hình thức.

LÀ

- Nhật ký các *quyết định quan trọng* khi làm việc với AI.
- Bảng chứng tư duy phản biện và tự chủ.
- Chứng minh sinh viên *hiểu* code và giải pháp, không phải copy.

Hình 7: AI Audit Log là gì và *không* là gì

7.2 Core Prompt vs. Auxiliary Prompt

Nguyên tắc vàng

Chỉ ghi những prompt mà **nếu không có nó, dự án sẽ KHÁC VỀ BẢN CHẤT** (kiến trúc, thiết kế, cách giải quyết) — không phải chỉ khác về syntax hay định dạng.

Loại	CORE — phải ghi	AUXILIARY — không ghi
Decision-Making	"So sánh dùng Map hay List để tra cứu theo ID?"	"Cách khai báo ArrayList trong Java?"
Problem-Solving	"Vì sao sort theo nhiều tiêu chí bị sai thứ tự?"	"Sửa lỗi thiếu dấu ; dòng 42."
Verification/Critique	"Regex <code>\d{6}</code> có bắt đúng 6 chữ số không, có thiếu <code>^\$</code> không?"	"Sinh getter/setter cho lớp Student."

Mẹo kiểm tra nhanh

Nếu *không* có prompt này mà dự án vẫn giống tới 90% → **LOẠI**. Nếu thiếu nó dự án khác về bản chất → **GHI**.

7.3 Yêu cầu số lượng (bài Assignment LAB211)

16-24

Core Prompts

≥ 2

Hallucination phát hiện

≥ 1

Core prompt / mỗi thành phần CT

Công thức gợi ý: **Số Core Prompts** = (số thành phần DTC) $\times$ (2-3 prompt/thành phần). Mỗi thành phần CT (Decomposition, Pattern Recognition, Abstraction, Algorithms) phải có *ít nhất một* core prompt tương ứng.

7.4 Cấu trúc một entry: 7 phần

Mỗi entry trong Audit Log gồm 7 phần; phần thứ 7 (**Human Delta**) là quan trọng nhất.

1. Entry #	Số thứ tự (001, 002, ...)
2. Prompt Type	DECISION / PROBLEM-SOLVING / VERIFICATION
3. Stage/Component	Decomposition, Pattern Recognition, Abstraction, Algorithms
4. Problem/Context	Vấn đề đang gặp (1-2 câu ngắn gọn)
5. Prompt to AI	Câu lệnh gửi cho AI (<i>nguyên văn</i>)
6. AI Response (Summary)	Tóm tắt phản hồi của AI (2-4 câu)
7. Human Delta & Reflection	Trả lời đủ 4 câu hỏi bên dưới

Bốn câu hỏi bắt buộc trong Human Delta

Critical Thinking

AI đúng/sai ở đâu? Có hallucination không?

Contextualization

Bối cảnh thực tế mà AI không biết?

Creative Synthesis

Tôi đã thay đổi/kết hợp/tối ưu thế nào?

Decision Ownership

Quyết định cuối cùng và lý do?

Hình 8: Bốn câu hỏi bắt buộc trong Human Delta

7.5 Phát hiện Hallucination (bắt buộc)

Hallucination là khi AI đưa ra thông tin *sai* nhưng trình bày rất tự tin. Năm loại phổ biến:

Fabrication	AI bịa thông tin không tồn tại (API/hàm/tham số không có thật).
Oversimplification	AI bỏ qua edge case (mảng rỗng, trùng dữ liệu, null).
Logic Error	Kết luận sai về "best practice" ("luôn dùng X là tốt nhất").
Outdated Info	Dùng cú pháp/API cũ đã deprecated.
Context Misunderstanding	Không hiểu đúng ràng buộc nghiệp vụ của đề bài.

Quy trình xử lý khi nghi ngờ AI sai

(1) Nêu nghi ngờ cụ thể → **(2)** Kiểm chứng độc lập (đọc tài liệu chính thống, chạy thử, viết test) → **(3)** Ghi lại *cách phát hiện* và *hành động sửa chữa* (corrective action).

7.6 Tiêu chí chấm AI Reflection (30%)

Phần AI Reflection chiếm **30%** tổng điểm, gắn vào ba mục tiêu (mỗi mục 10%): *AI Usage Documentation*, *Critical Thinking & Reflection on AI Output*, *Effective AI-assisted Learning Process*. Mỗi mục được chấm theo 4 mức.

Tiêu chí	Excellent (9–10)	Good (7–8)	Average (5–6)	Poor (0–4)
AI Usage Documentation	Ghi log đầy đủ; có prompt, kết quả AI, quyết định sử dụng/chỉnh sửa; rõ mục tiêu học tập.	Ghi phần lớn quá trình nhưng chưa đầy đủ, lập luận chưa thuyết phục/không nhất quán.	Có dùng AI nhưng log sơ sài, thiếu prompt hoặc thiếu giải thích (dùng mù quáng).	Không có AI Log hoặc chỉ copy output AI mà không giải thích.
Critical Thinking & Reflection	Phân tích đúng/sai dựa trên output AI; phát hiện lỗi; giải thích lý do chỉnh sửa; chứng minh hiểu mã nguồn.	Có phản biện kết quả AI nhưng chiều sâu phân tích chưa cao.	Chỉ mô tả AI hỗ trợ gì, ít phân tích đúng/sai.	Chấp nhận output AI thụ động; không chứng minh được hiểu biết.
Effective AI-assisted Learning	Dùng AI đúng vai trò: phân tích yêu cầu, thiết kế class, debug, refactor, giải thích thuật toán; có iterative prompting.	Dùng AI hợp lý nhưng chưa đa dạng hoặc thiếu chiều sâu.	Dùng AI chủ yếu để generate code cho xong việc.	Lạm dụng AI; không giải thích được sản phẩm hoặc quá trình học.

Pass/Fail trong Oral Vivas

Giảng viên sẽ hỏi vấn đáp 3–5 entries. Sinh viên **Pass** khi: giải thích được lý do quyết định cho $\geq 70\%$ entries được hỏi; có evidence cụ thể (code, metrics, so sánh); phát hiện ≥ 1 hallucination và giải thích cách fix; số entries nằm trong range. **Nếu FAIL ≥ 2 tiêu chí $\rightarrow$ AI Reflection = 0 điểm (mất 30%).**

7.7 Dấu hiệu nhận biết khi chấm

Dùng AI hiệu quả — sinh viên...	Dấu hiệu lạm dụng AI...
Biết chia nhỏ vấn đề; hỏi AI đúng trọng tâm.	Không giải thích được code; không hiểu logic.
Có iterative prompting; có debugging process.	AI log chỉ copy chat; không có reflection.
Có phản biện AI; có điều chỉnh AI output.	Không có decision making; code style không nhất quán.
Giải thích được code; liên hệ với nguyên lý OOP.	Không giải thích được tại sao chọn giải pháp đó.

7.8 Quy định cho sinh viên

Được phép (Allowed)	Không được phép (Not Allowed)
Giải thích kiến thức; học OOP; debug; refactor.	Nộp nguyên output AI mà không hiểu.
Gợi ý thuật toán; sinh code mẫu; giải thích lỗi.	Che giấu việc sử dụng AI.
Sinh test case.	Copy nguyên project từ AI; không giải thích được source code.

7.9 AI Literacy — viết prompt hiệu quả

Kỹ năng "Enhance AI Self Worth / AI Literacy" yêu cầu sinh viên hiểu nguyên tắc dùng GenAI, viết prompt rõ ràng, kiểm chứng thông tin và nắm rủi ro/đạo đức. Một prompt tốt thường có bốn thành phần:

Thành phần	Vai trò
Vai trò/Bối cảnh	"Tôi đang viết Java console theo OOP cho bài LAB211..."
Nhiệm vụ cụ thể	"...cần một Comparator sắp theo tên CLB rồi số áo."
Ràng buộc	"Không dùng thư viện ngoài; xử lý null an toàn."
Định dạng đầu ra	"Trả về đoạn code + giải thích từng bước."

Ví dụ iterative prompting (nhiều vòng)

Chuỗi prompt cải thiện dần

Vòng 1: "How to sort a list by two fields in Java?" → AI cho `Comparator.comparing().thenComparing()`.

Vòng 2 (thu hẹp): "What if some club names are null — will it throw NPE?" → AI gợi ý `Comparator.nullsLast()`.

Vòng 3 (kiểm chứng): "Show me a JUnit-style test proving order is correct for 3 sample items." → tôi chạy thử, xác nhận, và *tự quyết định* dùng phương án có null-safety.

7.10 Phương pháp Oral Vivas (vấn đáp)

Giảng viên đánh dấu 3-5 entries trong log và hỏi sâu khoảng 2-3 phút/entry. Bộ câu hỏi mẫu:

Loại prompt	Câu hỏi vấn đáp tiêu biểu
Decision-Making	"Tại sao em chọn approach B mà không phải A? Nhược điểm của B là gì? Em đã thử cả hai chưa, khác biệt bao nhiêu?"
Problem-Solving	"Lỗi gốc nằm ở đâu? Em đã khoanh vùng thế nào? Cách fix có tạo lỗi mới không?"
Verification	"Em phát hiện AI sai ở entry nào? Kiểm chứng bằng cách gì? Sau đó em làm gì?"

Dấu hiệu "log giả" (viết sau khi làm xong)

Sinh viên KHÔNG trả lời được vì sao chọn approach A thay vì B; KHÔNG nhớ AI đã trả lời gì; KHÔNG có evidence (ảnh chụp, diff code, số đo). Khi đó dù log "đẹp" vẫn bị đánh giá thấp.

7.11 Bảng phân bố entries theo thành phần CT

Khi nộp, sinh viên kèm bảng tóm tắt để giảng viên kiểm tra nhanh độ phủ:

Thành phần CT	Số entry	Yêu cầu	Loại prompt tiêu biểu
Decomposition	≥ 1	≥ 1	chia module, chọn cấu trúc dữ liệu
Pattern Recognition	≥ 1	≥ 1	regex, CRUD template
Abstraction	≥ 1	≥ 1	chọn thuộc tính/lớp, interface
Algorithms	≥ 1	≥ 1	sort, search, thống kê, file I/O
Tổng core prompts	16-24 (kèm ≥ 2 hallucination đã phát hiện)		

8

Ví dụ AI Audit Log cho Payroll

8.1 Entry ĐẠT — Decision (thiết kế kế thừa)

[✓] Entry #002 — DECISION-MAKING — Abstraction

Prompt to AI: "I have 4 employee roles with different allowance. Should I use one class with if-else on role, or inheritance?"

AI Response: gợi ý kế thừa + method override.

Human Delta: *Critical Thinking:* đồng ý; bổ sung lý do Open/Closed (thêm role không sửa vòng payroll). *Creative Synthesis:* tách `roleAllowance()` abstract, dùng `EmployeeFactory`. *Decision Ownership:* chốt 4 lớp con + factory; kiểm chứng bằng cách thêm thử role mới chỉ tốn một lớp.

8.2 Entry phát hiện HALLUCINATION (công thức lương)

Entry #011 — Hallucination: Fabrication / Context Misunderstanding

Prompt to AI: "What is the standard monthly payroll formula for this assignment?"

AI's Claim: AI khẳng định một công thức "chuẩn" (vd nhân hệ số thuế cố định) như thể đề quy định.

Reality Check: Đề không cho công thức → AI bịa "quy tắc chuẩn".

How Detected: Đọc kỹ đề; không có mục công thức; đối chiếu dữ liệu mẫu.

Corrective Action: Tự định nghĩa công thức minh bạch ($\text{baseSalary}/26 \times \text{workingDays} + \text{bonus} + \text{roleAllowance}$), ghi rõ giả định, chỉ tính active.

8.3 Entry ĐẠT — Problem-Solving (dòng dữ liệu lỗi)

[✓] Entry #014 — PROBLEM-SOLVING — Algorithms

Problem/Context: `employees.txt` có dòng thiếu trường / số sai khiến `parseDouble` ném exception.

Prompt to AI: "How to safely parse lines that may be malformed in Java without crashing?"

AI Response: gợi ý try-catch quanh parse.

Human Delta: *Creative Synthesis:* thêm kiểm tra đủ 7 trường + validate range + đếm & báo số dòng bỏ. *Decision Ownership:* chọn "skip + report" thay vì dùng toàn bộ; kiểm chứng với một dòng cố tình sai.

8.4 Entry KÉM (để đối chiếu)

[✗] Entry #008 — không đạt

Human Delta: "AI cho công thức lương, em dùng luôn."

Vì sao kém: không kiểm chứng (để không có công thức), không giải thích giả định, không evidence → **LOẠI** và là dấu hiệu dùng AI thụ động.

8.5 Ngân hàng câu hỏi vấn đáp (Payroll)

Chủ đề	Câu hỏi mẫu
Kế thừa	"Vì sao dùng 4 lớp con thay vì if-else theo role? Thêm role mới em sửa ở đâu?"
Đa hình	"calculateSalary() gọi roleAllowance() thế nào? Cho ví dụ Manager vs Tester."
Công thức lương	"Công thức của em là gì? Vì sao chia 26? Em kiểm chứng với dữ liệu nào?"
Payroll active	"Vì sao chỉ cộng nhân viên active? Nhân viên inactive xử lý ra sao?"
Load lỗi	"Khi gặp dòng hỏng em làm gì? Vì sao không để chương trình dừng?"

8.6 Chỉ mục Audit Log mẫu (21 core prompts)

Entry	Type	Component	Tóm tắt
#001	Decision	Decomposition	Tách model/business/tools + factory.
#002	Decision	Abstraction	Kế thừa thay if-else theo role.
#003	Decision	Abstraction	roleAllowance() abstract; interface Payable.
#004	Verification	Pattern Recognition	Regex E\d{3} cho ID.
#005	Problem-Solving	Pattern Recognition	Validate role/status bằng tập giá trị.
#006	Decision	Algorithms	Factory ánh xạ role → lớp con.
#007	Problem-Solving	Algorithms	Payroll chỉ cộng active.
#008	Problem-Solving	Algorithms	Search đa thuộc tính (ID/name/role/status).
#009	Problem-Solving	Abstraction	Update giữ field nếu Enter rỗng.
#010	Decision	Abstraction	Đổi role → tạo lại bằng factory.
#011	Verification	Algorithms	Hallucination: AI bịa "công thức lương chuẩn".
#012	Verification	Pattern Recognition	Hallucination: bỏ qua range workingDays 0-26.
#013	Problem-Solving	Algorithms	split có trim khoảng trắng quanh dấu phẩy.
#014	Problem-Solving	Algorithms	Load an toàn dòng lỗi (try-catch + đếm).
#015	Decision	Algorithms	Lưu text đúng format 7 trường.
#016	Problem-Solving	Algorithms	baseSalary phải dương; bonus ≥ 0.
#017	Problem-Solving	Abstraction	isActive() không phân biệt hoa thường.
#018	Decision	Algorithms	Định dạng cột printf cho bảng.
#019	Verification	Algorithms	Kiểm chứng tổng payroll khớp tính tay.
#020	Problem-Solving	Pattern Recognition	ID duy nhất khi Add & khi Load.
#021	Verification	Decomposition	Rà soát đủ 9 chức năng so với đề.

Phụ lục — Checklist tự chấm

#	Tiêu chí	Đạt?
1	9 chức năng chạy đúng	☐
2	Employee abstract + 4 lớp con + đa hình calculateSalary()	☐
3	Payroll chỉ tính nhân viên active	☐
4	load không crash với dòng lỗi, có báo cáo	☐
5	Công thức lương tự định nghĩa & giải thích (không bịa theo AI)	☐
6	16-24 core prompts, ≥ 2 hallucination (gồm công thức)	☐
7	Human Delta đủ 4 câu hỏi mỗi entry	☐

Lời kết

Payroll là đỉnh của ba đề: nó buộc sinh viên *thiết kế phân cấp lớp* và *đa hình* thực sự, *tự định nghĩa nghiệp vụ* (công thức lương) và *phản biện AI* khi AI bịa quy tắc. Làm tốt đề này nghĩa là đã làm chủ trọn vẹn vòng CT → OOP → AI Reflection.